

第十一章：特征选择与稀疏学习

—— 自强弘毅 求是拓新 ——

|| 目录 ||

CONTENTS

- 1 子集搜索与评价**
- 2 过滤式选择**
- 3 包裹式选择**
- 4 嵌入式选择与L1正则化**
- 5 稀疏表示与字典学习**
- 6 压缩感知**



01

子集搜索与评价



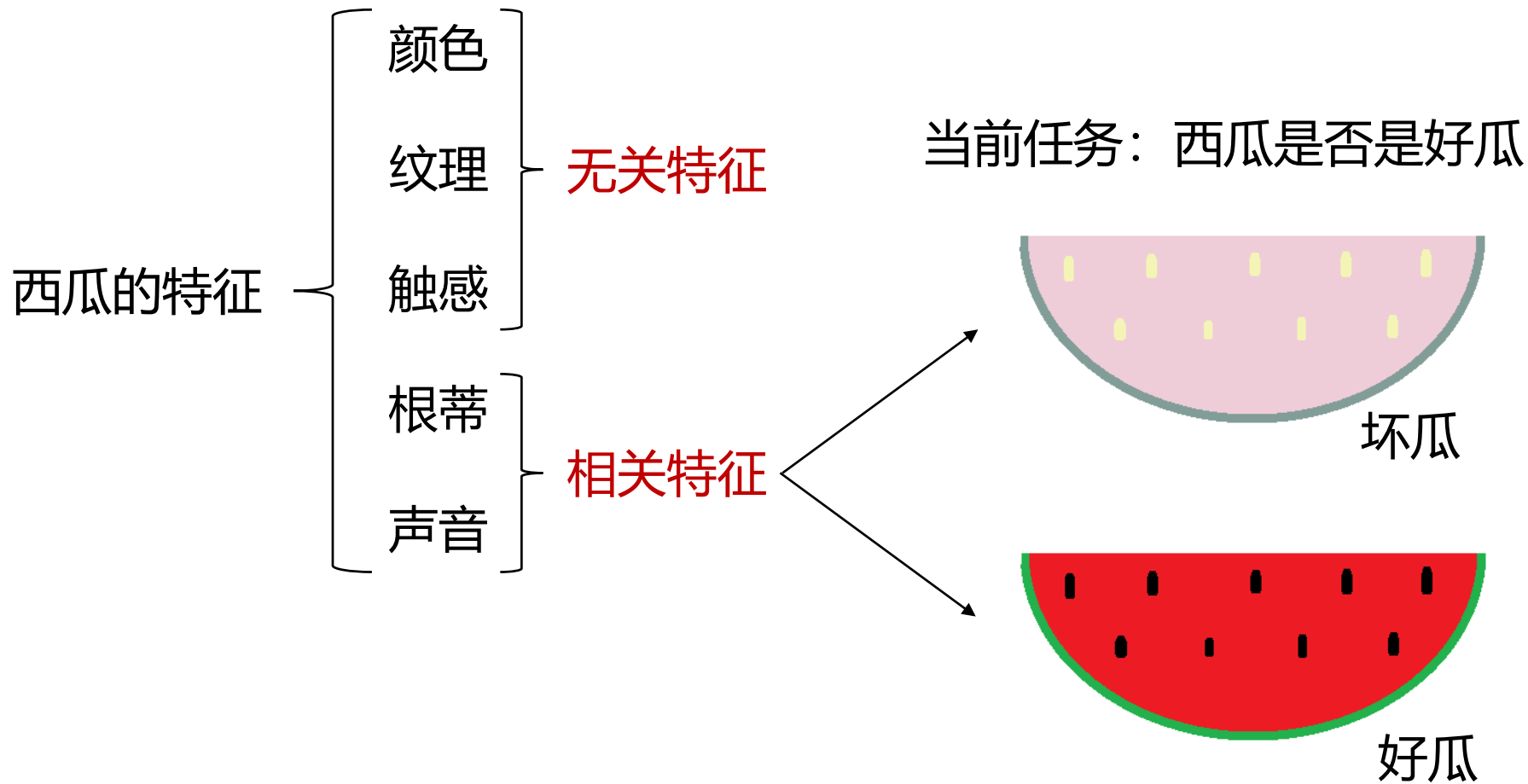
- **特征**：对象的（各种）属性描述，例如形状、颜色、大小、材质等
- 特征的分类
 - **相关特征**：对当前学习任务有用的属性
 - **无关特征**：与当前学习任务无关的属性
 - **冗余特征**：其所包含信息能由其他特征推演出来。若某个冗余特征对应学习任务的中间概念，则该冗余特征是有益的



例子：西瓜的特征



武汉大学
WUHAN UNIVERSITY





● 特征选择

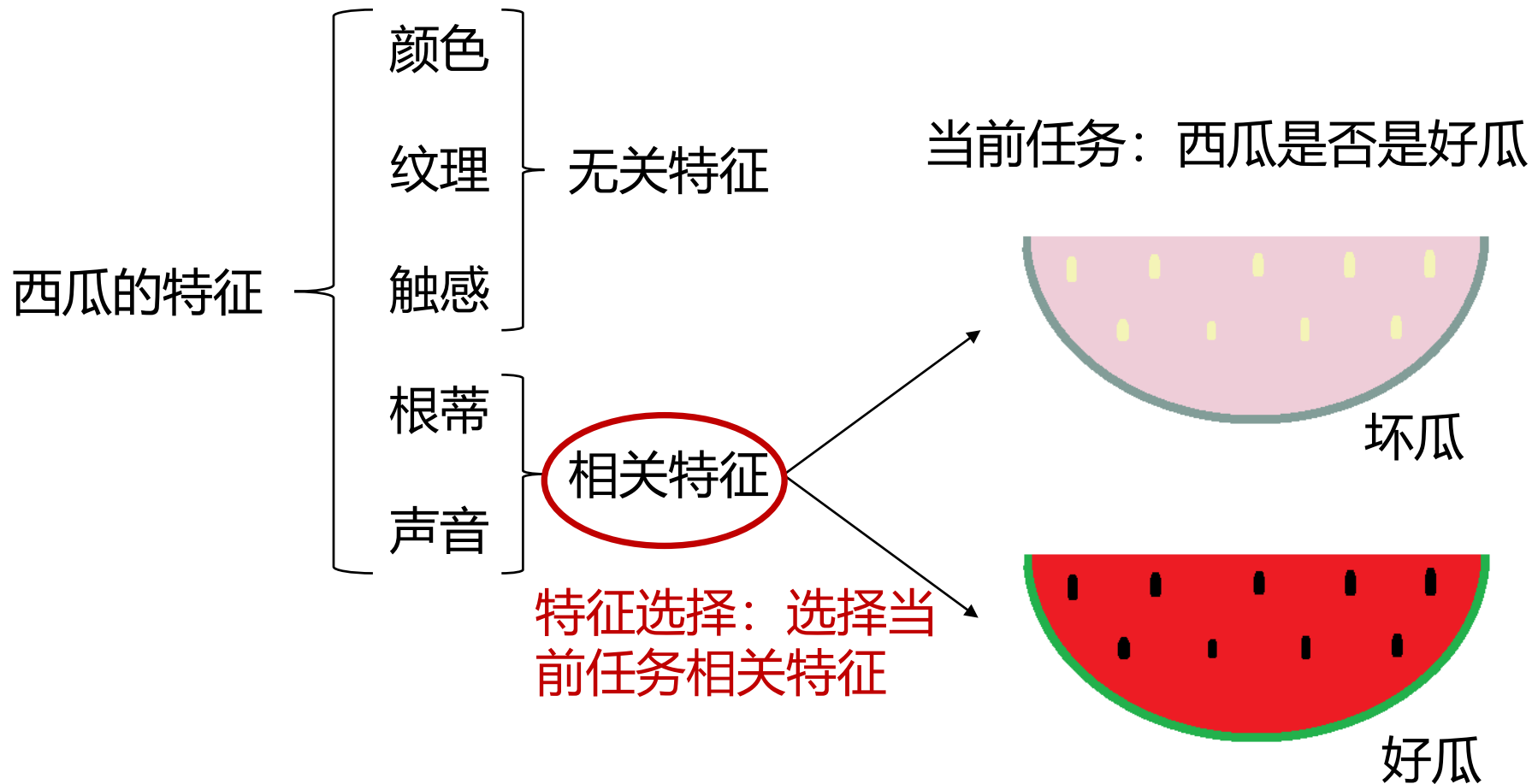
- 重要的**数据预处理**过程
- 从给定的特征集合中**选出任务相关特征子集**
- 必须确保**不丢失重要特征**
- 一般**先进行特征选择，再训练学习器**

● 原因

- **减轻维数灾难**：在少量属性上构建模型
- **降低学习难度**：去除不相关特征，留下关键因素，更易看清真相



例子：判断是否好瓜





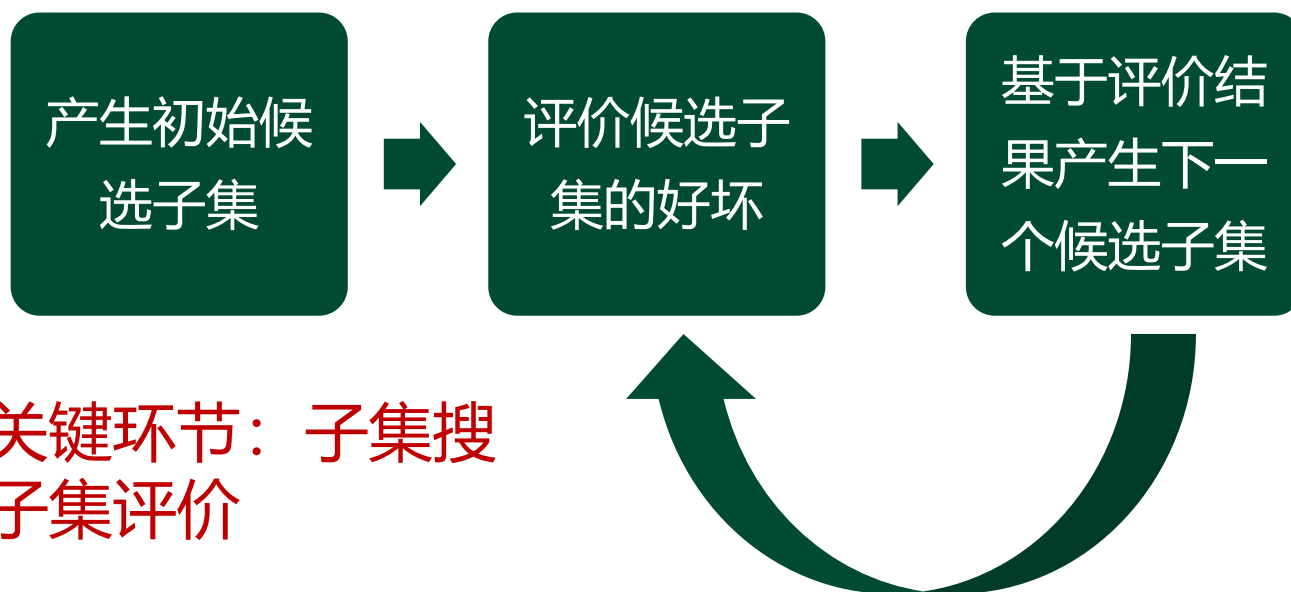
特征选择的一般方法



武汉大学
WUHAN UNIVERSITY

- 遍历所有可能的特征子集（无先验知识）
 - 计算上遭遇组合爆炸，不可行

● 可行方法



两个关键环节：子集搜索和子集评价



- **子集搜索** (subset search) **问题**: 给定特征集合 $\{a_1, a_2, \dots, a_d\}$, 我们可将每个特征看作一个候选子集, 对这 d 个候选单特征子集进行评价, 假定 $\{a_2\}$ 最优, 于是将 $\{a_2\}$ 作为第一轮的**选定集**
- 然后在上一轮的选定集中**加入**一个特征, 构成包含两个特征的候选子集, 假定在这 $d-1$ 个候选两特征子集中 $\{a_2, a_4\}$ 最优, **且优于** $\{a_2\}$, 于是将 $\{a_2, a_4\}$ 作为本轮的**选定集**



- 假定在第 $k + 1$ 轮时，最优的候选 $(k + 1)$ 特征子集不如上一轮的选定集，则停止生成候选子集，并将**上一轮**选定的 k 特征集合作为特征选择结果
- 这种**逐渐增加相关特征**的策略称为**前向** (forward) **搜索**
- 类似的，若我们从完整的特征集合开始，每次尝试去掉一个**无关特征**，这样逐渐减少特征的策略称为**后向** (backward) **搜索**



- 将前向与后向搜索结合起来，每一轮逐渐增加选定相关特征（这些特征在后续轮中将确定不会被去除）、同时减少无关特征，这样的策略称为**双向**（bidirectional）**搜索**
- **上述策略都是贪心的**，因为它们**仅考虑了使本轮选定集最优**，例如在第三轮假定选择 a_5 优于 a_6 ，于是选定集为 $\{a_2, a_4, a_5\}$ ，然而在第四轮却可能是 $\{a_2, a_4, a_6, a_8\}$ 比所有的 $\{a_2, a_4, a_5, a_i\}$ 都更优，遗憾的是，**若不进行穷举搜索，则这样的问题无法避免**



- **子集评价** (subset evaluation) 问题: 给定数据集 D , 假定 D 中第 i 类样本所占的比例为 p_i ($i = 1, 2, \dots, |\mathcal{Y}|$) 为便于讨论, 假定样本属性均为**离散型**。对属性子集 A , 假定**根据其取值**将 D 分成了 V 个子集 $\{D^1, D^2, \dots, D^V\}$, **每个子集中的样本在 A 上取值相同**, 于是我们可计算属性子集 A 的信息增益

$$\text{Gain}(A) = \text{Ent}(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Ent}(D^v)$$

- 其中**信息熵**定义为 $\text{Ent}(D) = - \sum_{i=1}^{|\mathcal{Y}|} p_k \log_2 p_k$



- 信息增益 $\text{Gain}(A)$ 越大，意味着特征子集 A 包含的有助于分类的信息越多，于是对每个候选特征子集，我们可基于训练数据集 D 来计算其信息增益，以此作为评价准则
- 更一般的，特征子集 A 实际上确定了对数据集 D 的一个划分，每个划分区域对应着 A 上的一个取值，而样本标记信息 Y 则对应着对 D 的真实划分，通过估算这两个划分的差异，就能对 A 进行评价。与 Y 对应的划分的差异越小，则说明 A 越好，信息熵仅是判断这个差异的一种途径，其他能判断两个划分差异的机制都能用于特征子集评价



- 将特征子集**搜索机制与子集评价机制相结合**，即可得到**特征选择方法**
- 例如将前向搜索与信息熵相结合，这与**决策树**算法非常相似，事实上决策树可用于特征选择，树结点的划分属性所组成的集合就是选择出的特征子集
- 其他的特征选择方法未必像决策树特征选择这么明显，但它们在本质上都是显式或隐式地**结合了某种（或多种）子集搜索机制和子集评价机制**



02

过滤式选择



- **过滤式方法**先对数据集进行特征选择，然后再训练学习器，特征选择过程**与后续学习器无关**。这相当于先用特征选择过程对初始特征进行**过滤**，再用过滤后的特征来训练模型
- **Relief** (Relevant Features) 是一种著名的过滤式特征选择方法，该方法设计了一个**相关统计量**来度量特征的重要性，下面介绍该方法



- Relief方法的**统计量是一个向量**，其每个**分量**分别对应于一个初始特征，而**特征子集的重要性**则是由子集中每个特征所对应的**相关统计量分量之和**来决定。于是最终只需指定一个**阈值 τ** ，然后选择比 τ 大的相关统计量分量所对应的特征即可；也可指定欲选取的特征个数 k ，然后选择相关统计量**分量最大的 k 个特征**
- Relief的**关键是如何确定相关统计量**



- 给定训练集 $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$, 对每个示例 $\mathbf{x}_i$, Relief先在 $\mathbf{x}_i$ 的同类样本中寻找其最近邻 $\mathbf{x}_{i, \text{nh}}$, 称为猜中近邻 (near-hit) , 再从 $\mathbf{x}_i$ 的异类样本中寻找其最近邻 $\mathbf{x}_{i, \text{nm}}$, 称为猜错近邻 (near-miss) , 相关统计量对应于属性 j 的分量为

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i, \text{nh}}^j)^2 + \text{diff}(x_i^j, x_{i, \text{nm}}^j)^2 \quad \text{式 (11.3)}$$

- 其中 x_a^j 表示样本 $\mathbf{x}_a$ 在属性 j 上的取值, $\text{diff}(x_a^j, x_b^j)$ 取决于属性 j 的类型, 离散型取值0或1, 连续型取为差的绝对值 (属性值已经规范到[0, 1])



- 从式 (11.3) 可看出, 若 x_i 与其猜中近邻 $x_{i,nh}$ 在属性 j 上的距离**小于** x_i 与其猜错近邻 $x_{i,nm}$ 的距离, 则说明属性 j 对区分同类与异类样本是有益的, 于是**增大**属性 j 所对应的统计量分量
- 反之, 若 x_i 与其猜中近邻 $x_{i,nh}$ 在属性 j 上的距离**大于** x_i 与其猜错近邻 $x_{i,nm}$ 的距离, 则说明属性 j 起负面作用, 于是**减小**属性 j 所对应的统计量分量。最后, 对基于不同样本得到的估计结果进行**平均**, 就得到各属性的**相关统计量**分量, 分量值越大, 则对应属性的分类能力就越强



- Relief是为二分类问题设计的，其扩展变体Relief-F能处理多分类问题
- 假定数据集 D 中的样本来自 $|\mathcal{Y}|$ 个类别，对示例 x_i ，若它属于第 k 类 ($k \in \{1, 2, \dots, |\mathcal{Y}|\}$)，则Relief-F先在第 k 类的样本中寻找 x_i 的最近邻示例 $x_{i, \text{nh}}$ 并将其作为猜中近邻，然后在第 k 类之外的每个类中找到一个 x_i 的最近邻示例作为猜错近邻，记为 $x_{i, l, \text{nm}}$ ($l = 1, 2, \dots, |\mathcal{Y}|; l \neq k$) 于是，相关统计量对应于属性 j 的分量为

$$\delta^j = \sum_i -\text{diff}(x_i^j, x_{i, \text{nh}}^j)^2 + \sum_{l \neq k} (p_l \times \text{diff}(x_i^j, x_{i, l, \text{nm}}^j)^2)$$

估计相关统计量只需在采样上进行即可



03

包裹式选择



- 与过滤式特征选择不考虑后续学习器不同，包裹式特征选择直接把最终将要使用的学习器的性能作为特征子集的评价准则。换言之，包裹式特征选择的目的是为给定学习器选择最有利于其性能、“量身定做”的特征子集
- 一般而言，由于包裹式特征选择方法直接针对给定学习器进行优化，因此从最终学习器性能来看，包裹式特征选择比过滤式特征选择更好；但另一方面，由于在特征选择过程中需多次训练学习器，因此包裹式特征选择的计算开销通常比过滤式特征选择大得多



- **LVW** (Las Vegas Wrapper) 是一个典型的包裹式特征选择方法，它在**拉斯维加斯方法** (Las Vegas method) 框架下使用**随机策略**来进行子集搜索，并以最终分类器的误差为特征子集评价准则

- **LVW算法描述**

输入: 数据集 D ;
特征集 A ;
学习算法 $\mathcal{L}$;
停止条件控制参数 T .

过程:

```
1:  $E = \infty$ ;  
2:  $d = |A|$ ;  
3:  $A^* = A$ ;  
4:  $t = 0$ ;  
5: while  $t < T$  do  
6:   随机产生特征子集  $A'$ ;  
7:    $d' = |A'|$ ;  
8:    $E' = \text{CrossValidation}(\mathcal{L}(D^{A'}))$ ;  
9:   if  $(E' < E) \vee ((E' = E) \wedge (d' < d))$  then  
10:     $t = 0$ ;  
11:     $E = E'$ ;  
12:     $d = d'$ ;  
13:     $A^* = A'$   
14:   else  
15:     $t = t + 1$   
16:   end if  
17: end while
```

输出: 特征子集 A^* .



- 上图算法第8行是通过在数据集 D 上，使用交叉验证法来估计学习器 $\mathcal{L}$ 的误差，注意这个误差是在**仅考虑特征子集 A'** 时得到的，即特征子集 A' 上的误差，若它比当前特征子集 A^* 上的误差更小，或误差相当但 A' 中包含的特征数更少，则将 A' 保留下来
- 需注意的是，由于LVW算法中特征子集搜索采用了**随机策略**，而每次特征子集评价都需训练学习器，计算开销很大，因此算法设置了停止条件控制参数 T 。然而整个算法是基于拉斯维加斯方法框架，**若初始特征数很多，则算法可能运行很长时间都达不到停止条件**



04

嵌入式选择与L1正则化



- 在过滤式和包裹式特征选择方法中，特征选择过程与学习器训练过程有明显的分别；与此不同，**嵌入式特征选择**是将**特征选择过程与学习器训练过程融为一体**，两者在同一个优化过程中完成，即在学习器训练过程中自动地进行了特征选择
- 给定数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ ，其中 $\mathbf{x} \in \mathbb{R}^d$ 并且 $y \in \mathbb{R}$ 。我们**考虑最简单的线性回归模型**，以平方误差为损失函数，则优化目标为

$$\min_{\mathbf{w}} \sum_{i=1}^m (y_i - \mathbf{w}^T \mathbf{x}_i)^2 \quad \text{式 (11.5)}$$



- 当样本特征很多，而样本数相对较少时，式 (11.5) 很容易陷入过拟合。为了缓解过拟合问题，可对式 (11.5) 引入正则化项，若使用 L_2 范数正则化，则有

$$\min_{\mathbf{w}} \sum_{i=1}^m (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2 \quad \text{式 (11.6)}$$

- 其中正则化参数 $\lambda > 0$ ，式 (11.6) 称为岭回归 (ridge regression)，通过引入 L_2 范数正则化，确能显著降低过拟合的风险



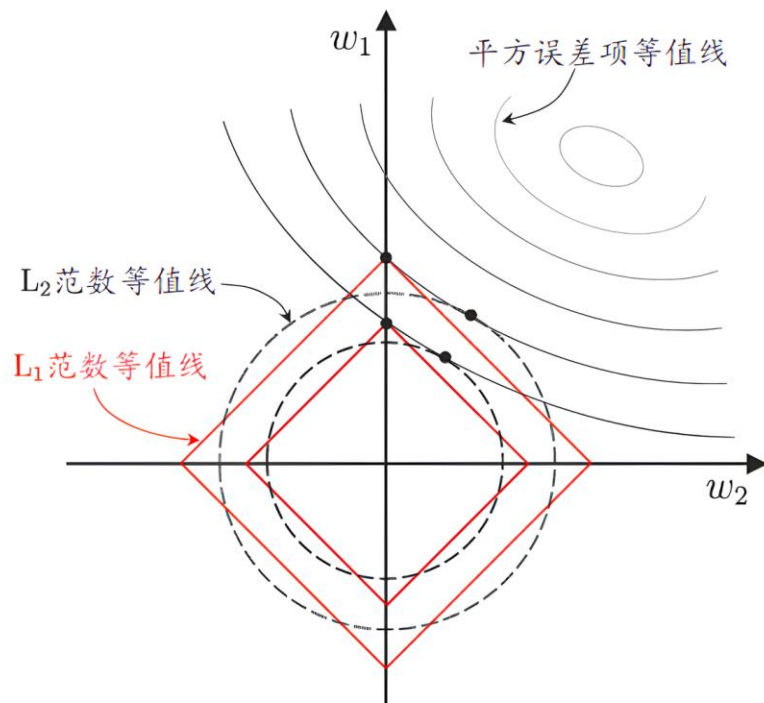
- 可以将正则化项中的 L_2 范数**替换**为 L_p 范数，若令 $p=1$ ，即采用 L_1 范数，则有

$$\min_{\mathbf{w}} \sum_{i=1}^m (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1 \quad \text{式 (11.7)}$$

- 其中正则化参数 $\lambda > 0$ ，上式称为**LASSO**
- L_1 范数和 L_2 范数正则化都有助于降低过拟合风险，但前者还会带来一个额外的好处，它比后者**更易于获得稀疏** (sparse) **解**，即它求得的 $\mathbf{w}$ 会有**更少的非零分量**



- 考虑 x 仅有**两个属性**，于是无论式 (11.6) 还是 (11.7) 解出的 w 都只有两个分量，即 w_1, w_2 ，我们将其作为两个坐标轴，然后在图中绘制出式 (11.6) 与 (11.7) 的第一项的**等值线**
- 由图可看出，采用 L_1 范数比 L_2 范数更易于得到**稀疏解**





- 注意到 w 取得稀疏解意味着初始的 d 个特征中仅有对应着 w 的非零分量的特征才会出现在最终模型中，于是求解 L_1 范数正则化的结果是得到了仅采用**一部分**初始特征的模型；换言之，**基于 L_1 正则化的学习方法就是一种嵌入式特征选择方法**，其特征选择过程与学习器训练过程融为一体，两者同时完成



- L_1 正则化问题的求解可使用近端梯度下降 (Proximal Gradient Descent, 简称 PGD), 具体来说, 令 ∇ 表示微分算子, 考虑如下优化目标

$$\min_{\mathbf{x}} f(\mathbf{x}) + \lambda \|\mathbf{x}\|_1 \quad \text{式 (11.8)}$$

- 若 $f(\mathbf{x})$ 可导, 且 ∇f 满足 L -Lipschitz 条件, 即存在常数 $L > 0$ 使得

$$\|\nabla f(\mathbf{x}') - \nabla f(\mathbf{x})\|_2^2 \leq L \|\mathbf{x}' - \mathbf{x}\|_2^2 \quad (\forall \mathbf{x}, \mathbf{x}')$$



- 则在 $\mathbf{x}_k$ 附近可将 $f(\mathbf{x})$ 通过二阶泰勒展开式近似为

$$\begin{aligned}\hat{f}(\mathbf{x}) &\simeq f(\mathbf{x}_k) + \langle \nabla f(\mathbf{x}_k), \mathbf{x} - \mathbf{x}_k \rangle + \frac{L}{2} \|\mathbf{x} - \mathbf{x}_k\|^2 \\ &= \frac{L}{2} \left\| \mathbf{x} - \left(\mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k) \right) \right\|_2^2 + \text{const}\end{aligned}$$

- 其中const是与 $\mathbf{x}$ 无关的常数, $\langle \cdot, \cdot \rangle$ 表示内积, 显然上式的最小值在如下 $\mathbf{x}_{k+1}$ 获得

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k)$$



- 若通过**梯度下降法**对 $f(\mathbf{x})$ 进行最小化, 则每一步梯度下降迭代实际上**等价于最小化二次函数** $\hat{f}(\mathbf{x})$, 将这个思想推广到式 (11.8), 则能类似地得到其每一步迭代应为

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \frac{L}{2} \|\mathbf{x} - \left(\mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k) \right)\|_2^2 + \lambda \|\mathbf{x}\|_1$$

即在每一步对 $f(\mathbf{x})$ 进行梯度下降迭代的同时考虑 L_1 范数最小化

- 对于上式, 可**先计算** $\mathbf{z} = \mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k)$, 然后求解

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \frac{L}{2} \|\mathbf{x} - \mathbf{z}\|_2^2 + \lambda \|\mathbf{x}\|_1 \quad \text{式 (11.13)}$$



- 令 x^i 表示 x 的第 i 个分量, 将式 (11.13) 按分量展开可看出, 其中不存在 $x^i x^j (i \neq j)$ 这样的项, 即 x 的**各分量互不影响**, 于是式 (11.13) 有**闭式解**

$$x_{k+1}^i = \begin{cases} z^i - \lambda/L, & \lambda/L < z^i; \\ 0, & |z^i| \leq \lambda/L; \\ z^i + \lambda/L, & z^i < -\lambda/L, \end{cases}$$

- 其中 x_{k+1}^i 与 z^i 分别是 x_{k+1} 与 z 的第 i 个分量, 因此, 通过PGD能使LASSO和其他基于 L_1 范数最小化的方法得以快速求解



05

稀疏表示与字典学习



- 不妨把数据集 D 考虑成一个矩阵，其**每行对应于一个样本，每列对应于一个特征**。特征选择所考虑的问题是特征具有“**稀疏性**”，即**矩阵中的许多列与当前学习任务无关**，通过特征选择去除这些列，则学习器训练过程仅需在较小的矩阵上进行，学习任务的难度可能有所**降低**，涉及的计算和存储开销会**减少**，学得模型的可解释性也会**提高**
- 考虑**另一种稀疏性**： D 所对应的矩阵中存在很多零元素，但这些零元素并不是以整列、整行形式存在的，在不少现实应用中我们会遇到这样的情形（文档分类）



- 当样本具有稀疏表达形式时，对学习任务来说会有不少好处，例如线性支持向量机之所以能在文本数据上有很好的性能，恰是由于文本数据在使用上述的**字频表示**后具有高度的稀疏性，使大多数问题变得**线性可分**
- 同时，稀疏样本并不会造成存储上的巨大负担，因为稀疏矩阵已有很多**高效的存储方法**
- 那么，若给定数据集 D 是**稠密的**，即**普通非稀疏数据**，能否将其**转化为稀疏表示**（sparse representation）形式，从而享有稀疏性所带来的好处呢



- 我们需学习出一个“字典”，为普通稠密表达的样本找到合适的字典，将样本转化为**合适的稀疏**表示形式，从而使学习任务得以简化，模型复杂度得以降低，通常称为**字典学习**（dictionary learning），亦称**稀疏编码**（sparse coding）
- 这两个称谓稍有差别，**字典学习**更侧重于学得字典的过程，而**稀疏编码**则更侧重于对样本进行稀疏表达的过程。由于两者通常是在同一个优化求解过程中完成的，因此下面笼统地称为**字典学习**



- 给定数据集 $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ ，字典学习最简单的形式为

$$\min_{\mathbf{B}, \alpha_i} \sum_{i=1}^m \|\mathbf{x}_i - \mathbf{B} \alpha_i\|_2^2 + \lambda \sum_{i=1}^m \|\alpha_i\|_1 \quad \text{式 (11.15)}$$

- 其中 $\mathbf{B} \in \mathbb{R}^{d \times k}$ 为字典矩阵， k 称为字典的词汇量，通常由用户指定， $\alpha_i \in \mathbb{R}^k$ 则是样本 $\mathbf{x}_i \in \mathbb{R}^d$ 的稀疏表示，显然式 (11.15) 的第一项是希望由 α_i 能很好地重构 $\mathbf{x}_i$ ，第二项则是希望 α_i 尽量稀疏
- 与LASSO相比，式 (11.15) 显然麻烦得多。但可以采用类似的变量交替优化策略求解



- 首先在第一步，固定住字典 $\mathbf{B}$ ，若将式 (11.15) 按分量展开，可看出其中不涉及 $\alpha_i^u \alpha_i^v$ ($u \neq v$) 这样的交叉项，于是可参照**LASSO**的解法求解下式，从而为每个样本 $\mathbf{x}_i$ 找到相应的 α_i

$$\min_{\alpha_i} \|\mathbf{x}_i - \mathbf{B} \alpha_i\|_2^2 + \lambda \|\alpha_i\|_1$$

- 在第二步，固定住 α_i 来更新字典 $\mathbf{B}$ ，此时可将式 (11.15) 写为

$$\min_{\mathbf{B}} \|\mathbf{X} - \mathbf{B} \mathbf{A}\|_F^2 \quad \text{式 (11.17)}$$

- 其中 $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m) \in \mathbb{R}^{d \times m}$, $\mathbf{A} = (\alpha_1, \alpha_2, \dots, \alpha_m) \in \mathbb{R}^{k \times m}$
 $\|\cdot\|_F$ 是矩阵的**Frobenius**范数



- 式 (11.17) 有多种求解方法，常用的有基于**逐列更新**策略的**KSVD**，令 $\mathbf{b}_i$ 表示字典矩阵 $\mathbf{B}$ 的第 i 列， α^i 表示稀疏矩阵 $\mathbf{A}$ 的第 i 行，式 (11.17) 可重写为

$$\begin{aligned}\min_{\mathbf{B}} \|\mathbf{X} - \mathbf{BA}\|_F^2 &= \min_{\mathbf{b}_i} \left\| \mathbf{X} - \sum_{j=1}^k \mathbf{b}_j \alpha^j \right\|_F^2 \\ &= \min_{\mathbf{b}_i} \left\| \left(\mathbf{X} - \sum_{j \neq i} \mathbf{b}_j \alpha^j \right) - \mathbf{b}_i \alpha^i \right\|_F^2 \\ &= \min_{\mathbf{b}_i} \|\mathbf{E}_i - \mathbf{b}_i \alpha^i\|_F^2\end{aligned}$$

式 (11.8)



- 在更新字典的第 i 列时，其他各列都是固定的，因此 $\mathbf{E}_i = \sum_{j \neq i} b_j \alpha^j$ 是固定的，于是最小化式 (11.8) 原则上只需对 $\mathbf{E}_i$ 进行奇异值分解以取得最大奇异值所对应的正交向量。然而，直接对 $\mathbf{E}_i$ 进行奇异值分解会同时修改 b_i 和 α^i ，从而可能破坏 $\mathbf{A}$ 的稀疏性
- 为避免发生这种情况，KSVD对 $\mathbf{E}_i$ 和 α^i 进行专门处理， α^i 仅保留非零元素， $\mathbf{E}_i$ 则仅保留 b_i 与 α^i 的非零元素的乘积项，然后再进行奇异值分解，这样就保持了第一步所得到的稀疏性
- 反复迭代上述两步，即可求得字典和样本的稀疏表示



06

压缩感知



- 在现实任务中，我们常**希望根据部分信息来恢复全部信息**，例如在数据通讯中要将模拟信号转换为数字信号，根据**奈奎斯特**（Nyquist）**采样定理**，令采样频率达到模拟信号最高频率的两倍，则采样后的数字信号就保留了模拟信号的全部信息；换言之，由此获得的数字信号能精确重构原模拟信号
- 然而，为了便于传输、存储，在实践中人们通常对采样的数字信号进行**压缩**，这有可能损失一些信息，而在信号传输过程中，由于信道出现**丢包**等问题，又可能损失部分信息



- 那么，接收方基于收到的信号，能否精确地重构出原信号呢？**压缩感知**（compressed sensing）为解决此类问题提供了新的思路
- 假定有长度为 m 的离散信号，不妨假定我们以远小于奈奎斯特采样定理要求的采样率进行采样，得到长度为 n 的采样后信号 $\mathbf{y}$, $n \ll m$ ，即 $\mathbf{y} = \Phi \mathbf{x}$
- 其中 $\Phi \in \mathbb{R}^{n \times m}$ 是对信号的**测量矩阵**，它确定了以什么频率进行采样以及如何将采样样本组成采样后的信号



- 在已知离散信号 x 和测量矩阵 Φ 时要得到测量值 y 很容易，然而，若将测量值和测量矩阵传输出去，接收方能还原出原始信号 x 吗？
- 一般来说，答案是 “No”，这是由于 $n \ll m$ ，因此 y ， x 以及 Φ 组成的 $y = \Phi x$ 是一个欠定方程，无法轻易求出数值解，也就是说还原不出原始信号了



- 现在不妨假设存在某个线性变换 $\Psi \in \mathbb{R}^{m \times m}$ 使得 x 可表示为 Ψs ，于是 y 可表示为 $y = \Phi \Psi s = A s$ 式 (11.20)
- 其中 $A = \Phi \Psi \in \mathbb{R}^{n \times m}$ ，于是，若能根据 y 恢复出 s ，则可通过 $x = \Psi s$ 来恢复出信号 x
- 粗看起来式 (11.20) 没有解决任何问题，因为式 (11.20) 中恢复信号 s 这个逆问题仍是欠定的。然而有趣的是，若 s 具有稀疏性，则这个问题竟能很好地得以解决！这是因为稀疏性使得未知因素的影响大为减少，此时式 (11.20) 中的 Ψ 称为稀疏基，而 A 类似于字典
- 压缩感知关注利用稀疏性，从部分观测样本恢复原始信号
- 压缩感知包括稀疏表示（傅里叶、小波、字典）与重构恢复两阶段



- **限定等距性** (Restricted Isometry Property, 简称 RIP) 是压缩感知理论中的一个重要概念, 用于描述一个矩阵在低维子空间上保留信号距离的能力, 是判断稀疏信号能否通过线性测量矩阵准确恢复的重要性质
- 对大小为 $n \times m$ ($n \ll m$) 的矩阵 $\mathbf{A}$, 若存在常数 $\delta_k \in (0, 1)$ 使得对于**任意**向量 $\mathbf{s}$ 和 $\mathbf{A}$ 的**所有**子矩阵 $\mathbf{A}_k \in \mathbb{R}^{n \times k}$ 有

$$(1 - \delta_k) \|\mathbf{s}\|_2^2 \leq \|\mathbf{A}_k \mathbf{s}\|_2^2 \leq (1 + \delta_k) \|\mathbf{s}\|_2^2$$

则称 $\mathbf{A}$ 满足 k 限定等距性 (k -RIP)



- 此时可通过下面的优化问题近乎完美地从 y 中恢复出稀疏信号 s ，进而恢复出 x

$$\min_s \|s\|_0, \text{ s.t. } y = As \quad \text{式 (11.22)}$$

- 然而，式 (11.22) 涉及 L_0 范数（非零元素个数）最小化，这是个NP难问题，值得庆幸的是， L_1 范数最小化在一定条件下与 L_0 范数最小化问题**共解**，于是实际上**只需关注**

$$\min_s \|s\|_1, \text{ s.t. } y = As \quad \text{式 (11.23)}$$

- 这样，**压缩感知问题就可通过 L_1 范数最小化问题求解，将其转化为LASSO的等价形式通过近端梯度下降法求解**



- **矩阵补全** (matrix completion) 技术是通过**观察到的矩阵部分已知值来推测未知值**的一种技术, 这种方法在大规模数据分析中被广泛应用, 尤其是在数据稀疏性较高的场景下
- 矩阵补全的形式为

$$\begin{aligned} & \min_{\mathbf{X}} \text{rank}(\mathbf{X}) \\ & \text{s.t. } (\mathbf{X})_{ij} = (\mathbf{A})_{ij}, \quad (i, j) \in \Omega \end{aligned} \quad \text{式 (11.24)}$$

- 其中, $\mathbf{X}$ 表示需恢复的稀疏信号, $\text{rank}(\mathbf{X})$ 表示矩阵 $\mathbf{X}$ 的秩, $\mathbf{A}$ 是已观测信号, Ω 是 $\mathbf{A}$ 中确定元素的下标集合。上式是NP难问题。



- 注意到 $\text{rank}(\mathbf{X})$ 在集合 $\{\mathbf{X} \in \mathbb{R}^{m \times n} : \|\mathbf{X}\|_F^2 \leq 1\}$ 上的凸包是 $\mathbf{X}$ 的核范数 (nuclear norm)

$$\|\mathbf{X}\|_* = \sum_{j=1}^{\min\{m,n\}} \sigma_j(\mathbf{X})$$

- 其中 $\sigma_j(\mathbf{X})$ 表示 $\mathbf{X}$ 的奇异值，即矩阵的核范数为矩阵的奇异值之和，于是可通过最小化矩阵核范数来近似求解式 (11.24)。**以下问题可通过半正定规划方法求解**

$$\begin{aligned} & \min_{\mathbf{X}} \|\mathbf{X}\|_* \\ & \text{s.t.} \quad (\mathbf{X})_{ij} = (\mathbf{A})_{ij}, \quad (i,j) \in \Omega \end{aligned}$$



武汉大学
WUHAN UNIVERSITY

谢谢

自强弘毅求是拓新

课后阅读西瓜书第11章